

Warp Modularity

From paper IORs to a code-level interface

Internal note

A single interface every Warp phase satisfies, that is visibly the same shape as the paper's IOR signature.

- `lib.rs::prove` reads as a chain of typed transitions.
- Each transition: $(\text{stmt}, \text{wit}, \text{or}_{\text{in}}) \mapsto (\text{stmt}', \text{or}_{\text{out}})$.
- Adding a phase: a struct + a trait impl. No bespoke plumbing.

Why Warp wasn't easy modularly

The paper IOR formalism doesn't share state across IORs. Real Warp does.

Three concrete frictions:

1. **Shared oracle f .** TwinConstraint, OOD, Batching, Proximity all consume the same codeword. A literal IOR-per-phase would rematerialise.
2. **Transcript ordering coupling.** Shift-query indices are squeezed once and consumed by both Batching and Proximity. No phase “owns” the squeeze.
3. **Statement / witness / oracle conflation.** The pre-refactor function had ~ 10 mixed-purpose args; the paper's tripartite split was invisible.

What an IOR signature is

The Warp paper structures the protocol as a sequence of Interactive Oracle Reductions, each one having the composition rule:

$$A : (\text{stmt}_A, \text{wit}_A) \mapsto (\text{stmt}'_A, \text{O}[f])$$

$$B : (\text{stmt}_B, \text{O}[f]) \mapsto \text{stmt}'_B$$

$$A; B : (\text{stmt}_A, \text{stmt}_B, \text{wit}_A) \mapsto (\text{stmt}'_A, \text{stmt}'_B)$$

Five typed components per IOR:

	Prover	Verifier
Statement	✓	✓
Witness	✓	—
InputOracles	by data	by handle
ReducedStatement	✓	✓
OutputOracles	by data	by handle

The IOR trait we built

Current shape (post-PR#22 review) mirrors the paper and routes both sides through one reduction function:

```
pub trait IOR {  
    type Statement; type Witness;  
    type ProverInputs;    type VerifierInputs;  
    type ReductionInputs;  
    type ReducedStatement;  
    type ProofString;    type ReducedWitness;  
    type VerifierOutputs;  
  
    fn reduce_statement(&self, &Stmt, &ReductionInputs) -> Reduced;  
    fn prove_inner(...) -> (ReductionInputs, ProofString, ReducedWitness);  
    fn verify_inner(...) -> (ReductionInputs, VerifierOutputs);  
    // default 'prove' / 'verify' chain *_inner -> reduce_statement  
}
```

Implementor writes the three; defaults wire them. Statement / ReducedStatement shared; oracle types role-split; reduction is single-source-of-truth. 33/33 tests green.

Per-phase mapping

	Stmt	Wit	Pr. In	Reduced	Proof	Red. Wit
Pesat	$(l_1, \log m)$	wits	—	(μ, τ)	—	cw + tree
TwinC.	$\text{acc} + \mu/\tau$	acc-w + ins	cw + acc-cw	$\gamma, \zeta_0, \beta_\tau,$ defer	—	$O[f] + z$
Ood	$(s, \log n)$	—	$O[f]$	samples + ans	—	—
Batching	$\zeta + s, t, \log n$	—	$O[f]$	α	—	μ
Proximity	queries + l_2, t	—	trees + cw	$()$	paths + ans	—

Asymmetries are now structural:

- Pesat — empty ProverInputs (it's the source).
- Proximity — empty Reduced and Red. Wit (it's a check; pure ProofString).
- Ood / Batching — empty Witness.
- ProofString is non-empty only for Proximity; every other phase's prover output is reduced-witness-shaped.

Single-protocol view: a candid review

What's good.

- Paper-aligned types — the IOR trait and paper composition rule line up directly.
- Verifier symmetry — `*::verify` are first-class trait impls.
- Honest oracle role split — prover sees data, verifier sees *handles* (see oracle slide).
- Empty-type asymmetries visible (`ProofString = ()` for reductions; `Reduced = ()` for checks).
- Statement reduction is single-source-of-truth — no prover/verifier drift.

What's not.

- Nine associated types per phase — real cognitive load.
- Lifetimes & `PhantomData` proliferate; structs carry `<'a, F, ...>`.
- No type-level ordering enforcement; transcript order is implicit.
- Orchestrator got longer, not shorter — ~ 10 lines per phase, plus oracle-handle construction.

Reframing: a family of protocols

The single-protocol view above was honest — but the actual goal isn't “a clean WARP.”

We design protocols like **STIR**, **WHIR**, **WARP** all the time. They share:

- Encode-and-commit (codeword — Reed-Solomon, or otherwise).
- Sumcheck reductions (different flavours: degree-1, multilinear, fused twin-constraint, . . .).
- Out-of-domain (OOD) sampling for proximity boost.
- Shift-query opening with auth paths.
- Batching sumchecks that fold many claims into one.
- Accumulator updates / final consistency.

The bar for the IOR trait then shifts: not “does this clean up WARP?” but “is this the right backbone for a family?”

The toolkit, audited

What practitioners writing STIR / WHIR / WARP *already* have:

- **Transcripts.** `spongefish 0.7` — de facto in this corner of the ecosystem. Practitioners use it directly. No abstraction needed.
- **Sumchecks.** `effsc` — `SumcheckProver`, `RoundPolyEvaluator`, `InnerProductProver`, `CoefficientProverLSB`, `MultilinearProver`, `hypercube` + `protogalaxy` utilities. WARP uses 9 of 10 `effsc` primitives already.
- **Linear codes.** `ark-codes::LinearCode` — `encode`, `code.len`.
- **Vector commitments.** `ark-vc` (abstract trait) + `ark-mt` (Merkle backend, multi-vector + single-shot openings, Blake3 / Poseidon hashers).
- **Oracle struct.** `Vec<F>` codeword + lazy MLE cache. Same storage shape across STIR / WHIR / WARP.

The primitive layer is mostly done. The discussion is really about *integration* — how primitives compose, and what shape protocol authors copy.

What practitioner experience actually needs

Designing a STIR / WHIR / WARP-style protocol is hard. The friction is rarely “I need a new sumcheck primitive.” It’s:

- **Tying primitives together correctly.** LinearCode + Merkle is two crates; integration is per-protocol boilerplate.
- **Knowing the right phase decomposition.** What’s a phase? What’s its statement? Where’s the witness? When does the verifier act?
- **Threading transcript ordering without bugs.** Squeeze before write, derive challenges in the right segments.
- **Maintaining paper-code alignment.** The paper’s IOR composition is the contract; reviewers expect to see it in code.
- **Debugging when soundness fails.** “Where in this 600-line `verify` did the rejection happen?”

The fellow: *practitioners benefit from patterns + examples + shared primitives*, not from cross-protocol trait contracts. That’s the lever.

What WARP migrated onto: ark-vc / ark-mt

Replaced `ark-crypto-primitives::merkle_tree` with a multi-vector commitment scheme. Two structural changes:

1. Per-codeword $\rightarrow$ one tree. PESAT commits l_1 fresh codewords; leaves are column-tuples $(c_0[i], \dots, c_{l_1-1}[i])$, hashed as `Vec<F>`. One root authenticates all codewords.

2. Per-query $\rightarrow$ one proof. Was t separate `Path<MT>`, each carrying $\log n$ siblings. Now a single path-pruned `OpeningProof` for all t positions (union of copaths minus union of paths, book §20).

At $n=2^{15}$, $t=100$: 1500 $\rightarrow \approx 730$ sibling hashes. Savings grow with t .

Type diff: `MerkleTree<MT>  $\rightarrow$  MultiVectorCommitted<H,S,F>; Path<MT>  $\rightarrow$  OpeningProof<HashRegion<H>>; H: MerkleHasher<Symbol=Vec<F>>` replaces MT: Config across all phases.

What we're for, positively

Keep IOR as a structural template, not a cross-protocol contract.

- It's the shape WARP-style protocols copy. The paper section on accumulator-as-state makes it legible.
- Other protocols follow as a *pattern*, get shared mental model + idioms, without being forced into one trait.

Lean on the existing primitive layer.

- `effsc`, `spongefish`, `ark-codes`, `ark-vc` + `ark-mt`.
- Already strong. The discipline is to use them, not reinvent.

WARP runs on `ark-vc`. The missing integration trait was filled by `ark-vc/ark-mt`; WARP migrated onto its multi-vector commitment scheme. STIR-lite plugs into the same surface.

Make WARP the reference implementation. A new protocol designer reads `src/protocol/phases/`, copies the IOR-shaped pattern, plugs in their primitives.

Document the pattern. Short “how to design a Warp-style protocol” guide — narrative, not a trait. The actual lever for practitioner ergonomics.

reduce_statement — eliminating prover/verifier drift (1 of 2)

Problem. Prover and verifier each compute `ReducedStatement` via parallel-but-different code; they produce the same value by sumcheck-fold semantics, nothing structural keeps them in sync. Concrete site: `TwinConstraint's  $\zeta_0$  /  $\beta_\tau$` — prover pulls from sumcheck CC's `internal.tablewise()`, verifier recomputes via `scale_and_sum`. Patch one, forget the other, ship divergence.

Change.

```
fn reduce_statement(&self, &Stmt, &ReductionInputs) -> ReducedStatement;  
fn prove_inner(...) -> (ReductionInputs, ProofString, ReducedWitness);  
fn verify_inner(...) -> (ReductionInputs, VerifierOutputs);  
// default 'prove'/'verify' chain *_inner -> reduce_statement.
```

Both sides land at `reduce_statement` with the same `ReductionInputs`; the math runs in one place.

Outcome. Drift is structurally impossible: prover pulls only f, z from CC; verifier stops parallel `scale_and_sum`.
Cost: 1 associated type + 2 methods. 33/33 tests green.

IndexedOracle — BCS-agnostic verifier-side oracles (2 of 2)

Problem. `ProximityVerifierInputs` carried `{ rt_0, l2_roots, auth_0, auth_j, shift_query_answers }` — a fully materialised BCS instantiation glued onto the IOR-level type. Slide 4 promises “by handle”; the code delivered “by precomputed table plus auth artifacts.” Phase verifier knew about Merkle paths.

Change.

```
trait IndexedOracle<A> { fn query(&self, i: usize) -> Option<A>; ... }  
struct MerkleIndexedOracle<'a, F, H> { /* root, proof, idx+vals, OnceCell */ }  
impl IndexedOracle<Vec<F>> for MerkleIndexedOracle<...> { ... }
```

`ProximityVerifierInputs` is now generic over `O`: `IndexedOracle<Vec<F>>` and holds `&O / &[O]` (static dispatch, no trait objects). BCS handle construction moves to the orchestrator.

Outcome. Phase code is BCS-agnostic — two `validate()` calls in `verify_inner`. iBCS / ARG composition slots in via a different impl. Lazy validation `OnceCell`-memoised, so perf matches the prior eager check.

Caveat. Prover-side `Oracle<F>` (OOD / Batching) is still concrete; sibling `EvalOracle<F>` for point queries is a follow-up.

Where this leaves us — the next move

Already landed. ark-vc/ark-mt integration ($\sim 16\%$ smaller proof, prover matches with parallel). Four PR-#22 follow-ups: &-borrow, ProofString/ReducedWitness split, IndexedOracle handle, reduce_statement split. Tests + benches green.

Next.

- Paper section — accumulator as typed state: IOR trait as paper-aligned *template* for WARP-family protocols.
- Paper section — structured sumcheck primitive: TwinConstraint's fused $\alpha/\beta/\tau$ -fold extracted into effsc as a citable RoundPolyEvaluator impl.
- Sibling EvalOracle<F> — point-query analogue of IndexedOracle so prover-side OOD / Batching is BCS-agnostic too.
- Second protocol — STIR-lite or WHIR-lite against the same primitives. Friction it surfaces is the next iteration's guide.
- Practitioner narrative — “how to design a Warp-style protocol,” docs/.